

LeetCode編碼：

682

系所：北聯(北醫) 生物醫學工程學系

學號：B812111001

姓名：張博翔



題目 (含解釋&說明)



- 給定一字串陣列 `ops`，模擬棒球比賽計分過程。
陣列中的每個元素代表一種操作，共有四種可能：
整數字串代表這輪的得分直接記錄；
'+' 代表這輪得分為前兩輪得分之和；
'D' 代表這輪得分為前一輪得分的兩倍；
'C' 代表取消前一輪的得分並從記錄中移除。所有操作執行完後，回傳所有有效得分的總和。
- 以 `ops = ["5", "2", "C", "D", "+"]` 為例。
第一步記錄 5，有效記錄為 [5]。
第二步記錄 2，[5, 2]。
第三步 'C' 取消前一輪移除 2，變回 [5]。
第四步 'D' 記錄前一輪的兩倍， $5 \times 2 = 10$ ，[5, 10]。
第五步 '+' 記錄前兩輪之和， $5 + 10 = 15$ ，[5, 10, 15]。最後總和為 $5 + 10 + 15 = 30$ 。

題目保證 '+' 操作時記錄中至少有兩筆有效得分、'D' 與 'C' 時至少有一筆，因此不需要自行處理記錄不足的防錯邏輯。



- 解法一： **Stack**
- 觀察四種操作的共同特性：每一種操作都只需要存取最近的一到兩筆記錄，可運用**stack**特性，存取最近的記錄就是存取 **stack top**，不需要任何索引計算。
- 每個操作對應一種 **stack** 行為：
整數直接 **push**；
'+' 讀取頂端兩個值相加後 **push**（不移除原本的值，因為前兩輪得分依然有效）；
'D' 讀取頂端值乘以 2 後 **push**；
'C' 直接 **pop** 移除頂端。所有操作執行完後，**stack** 中剩下的就是所有有效得分，直接 **sum(stack)** 即為答案。
- 每個操作與 **stack** 行為的對應關係直觀。缺點是需要額外的 $O(n)$ 空間存放 **stack**，但由於 **stack** 的大小不超過操作數，這在本題是還可以接受的。



- 解法二：In-place 陣列指標
- 思路與解法一相同，差別在於不另外建立 `stack` 物件，而是預先分配一個與 `ops` 等長的陣列，用整數指標 `top` 模擬 `stack` 的頂端位置。
`top` 指向下一個可寫入的位置，`push` 等同於在 `scores[top]` 寫入值後將 `top += 1`，`pop` 等同於將 `top -= 1`。
 省去了 Python list 在動態 `append` 與 `pop` 時的物件管理。Python list 在 `append` 時若超過當前容量會觸發記憶體重新分配，而預先分配固定大小陣列能避免此問題。
 最後計算總和時只需 `sum(scores[:top])` 取有效範圍即可。
- 缺點是可讀性低於前一解法，`top` 指標的語意需要理解為下一個可寫入位置而非當前最後一個有效位置，容易造成 `off-by-one` 混淆。
 兩種方法的時間與空間複雜度完全相同，因此傾向優先選擇可讀性較高的 `Stack`。



- 每個操作只依賴最近的歷史記錄，直覺選擇 **Stack**。
當一個問題的每一步只需要存取最新的幾個狀態，且狀態可以被取消或覆蓋時，可以判斷是 **Stack** 的應用。
在 **LC 20**、**LC 739** 等題目中也有出現相似概念。

方法	Time	Space
Stack	$O(n)$	$O(n)$
In-place	$O(n)$	$O(n)$



```

1 class Solution(object):
2     # 解法一：Stack 模擬
3     def calPoints_stack(self, ops):
4         stack = []
5         for op in ops:
6             if op == '+':
7                 # 前兩輪得分仍然有效，不能 pop，只讀取不移除
8                 stack.append(stack[-1] + stack[-2])
9             elif op == 'D':
10                stack.append(stack[-1] * 2)
11            elif op == 'C':
12                stack.pop()
13            else:
14                stack.append(int(op))
15        return sum(stack)

```

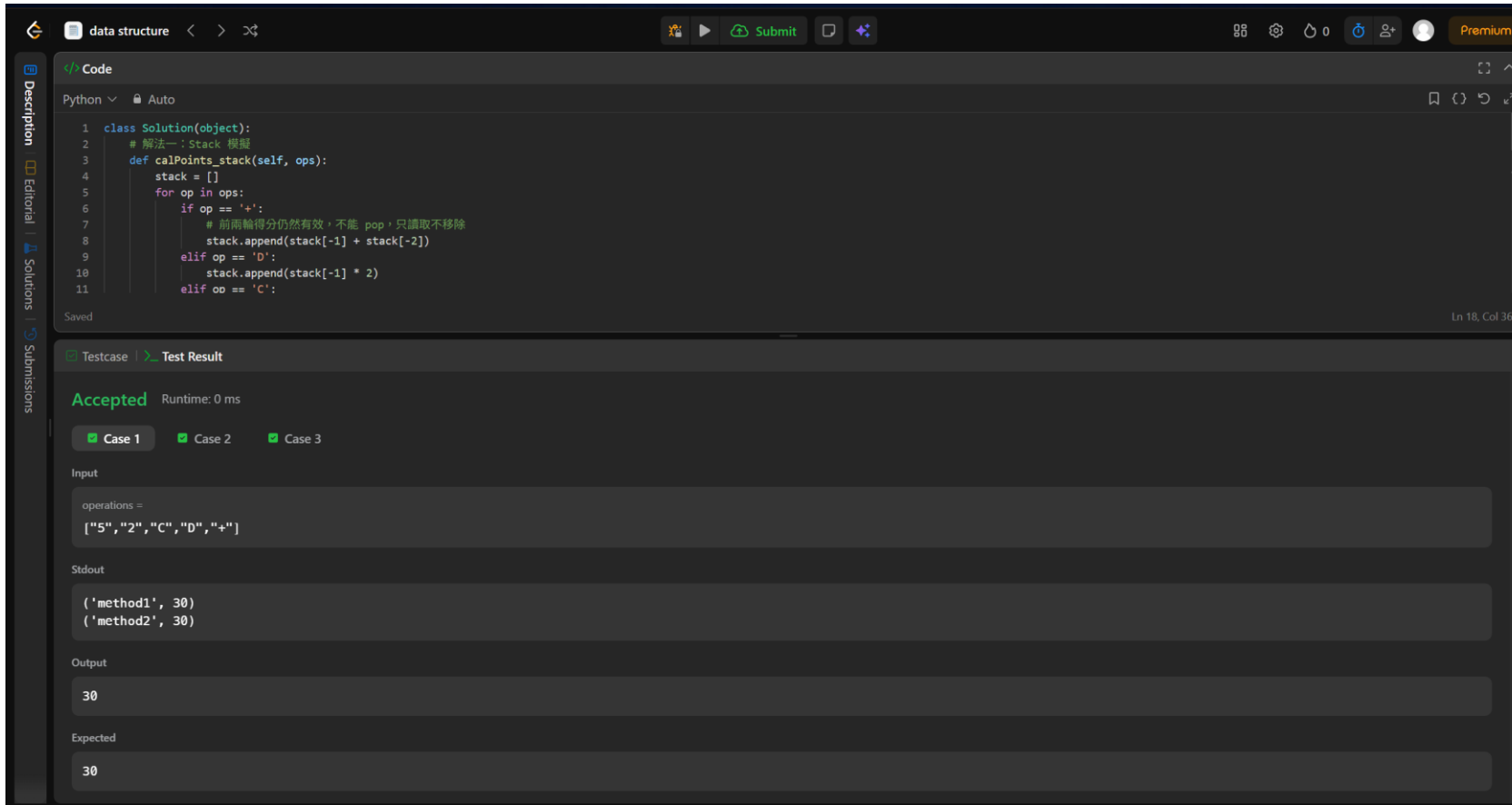
```

17 # 解法二：In-place 陣列指標
18 def calPoints_inplace(self, ops):
19     # 預先分配最大可能大小，避免動態 append 觸發記憶體重新分配
20     scores = [0] * len(ops)
21     # top 指向下一個可寫入的位置，等同於 stack 的頂端加一
22     top = 0
23     for op in ops:
24         if op == '+':
25             scores[top] = scores[top-1] + scores[top-2]
26             top += 1
27         elif op == 'D':
28             scores[top] = scores[top-1] * 2
29             top += 1
30         elif op == 'C':
31             # top 退一格，舊值留在陣列中但視為無效
32             # 下次寫入時自然覆蓋，不需要清除
33             top -= 1
34         else:
35             scores[top] = int(op)
36             top += 1
37     # 只取 top 以內的有效範圍加總
38     return sum(scores[:top])
39
40 def calPoints(self, operations):
41     res1 = self.calPoints_stack(operations)
42     res2 = self.calPoints_inplace(operations)
43
44     print("method1", res1)
45     print("method2", res2)
46
47     return res1

```



- 結果證明：



```

1 class Solution(object):
2     # 解法一：Stack 模擬
3     def calPoints_stack(self, ops):
4         stack = []
5         for op in ops:
6             if op == '+':
7                 # 前兩輪得分仍然有效，不能 pop，只讀取不移除
8                 stack.append(stack[-1] + stack[-2])
9             elif op == 'D':
10                stack.append(stack[-1] * 2)
11            elif op == 'C':

```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

```
operations =
["5", "2", "C", "D", "+"]
```

Stdout

```
('method1', 30)
('method2', 30)
```

Output

```
30
```

Expected

```
30
```



LeetCode編碼：

3412

系所： 北聯(北醫) 生物醫學工程學系
學號： B812111001
姓名： 張博翔





給定一個只包含小寫英文字母的字串 `sazbycx`

- 以 `s = "aczzx"` 為例。
索引 0 的 `a` 出現，找鏡像 `z`，目前沒有未配對的 `z`，記錄 `a` 的索引 0 等待。
索引 1 的 `c` 出現，找鏡像 `x`，目前沒有未配對的 `x`，記錄 `c` 的索引 1 等待。
索引 2 的 `z` 出現，找鏡像 `a`，發現索引 0 的 `a` 正在等待，配對成功，得分 $2 - 0 = 2$ ，將該 `a` 從等待名單中移除。
- 需要注意同一個字母可能出現多次，每次只能與最近一個未配對的鏡像字元配對而不是任意。
如連續出現兩個 `a` 再出現兩個 `z`，第一個 `z` 應配對最後出現的 `a`（索引差最小），因此每個字母需要獨立維護一個 `stack` 來記錄等待配對的索引，確保每次取出的都是最近出現。



- 解法一：HashMap + 各字母獨立 Stack
- 核心為每個字母各自維護一個 stack，用 defaultdict(list) 讓每個字母的索引按照出現順序堆疊，每次配對時從 stack 頂端取出最近的索引。
選擇 stack 而非 queue 是因為最近出現的優先配對的語意對應 stack 的 LIFO，若使用 queue 則會讓最早出現的先配對(FIFO)，不符。
- 遍歷字串時，對每個字元計算其鏡像字元，然後查看鏡像字元的 stack 是否有等待的索引。若有，pop 出最近的索引，計算與當前索引的差值加入總分；若沒有，將當前索引 push 進 stack 等待配對。
- 使用 HashMap 是為了讓每個字母的 stack 都能在 $O(1)$ 時間內被存取，無論是查詢鏡像字元的 stack 還是將當前字元推入自己的 stack，都不需要線性搜尋。
整體時間複雜度為 $O(n)$ ，空間複雜度為 $O(n)$ （最差情況下所有字元都未配對）。



- 解法二： **Stack + 線性搜尋**
- 用一個共同 **stack** 存放所有未配對字元的 (字元, 索引)，每次遇到新字元時，從 **stack** 頂端往下線性搜尋是否有鏡像字元。若找到則配對計分並從 **stack** 中移除，若沒找到則 **push** 進 **stack**。
- 問題在於每次搜尋都需要遍歷整個 **stack**，時間複雜度為 $O(n)$ 每步，整體為 $O(n^2)$ 。當字串很長時這個做法超時。

純粹是作為對比，展示為什麼需要 **HashMap**，當每次查找需要 $O(n)$ 時整體效率大幅下降；而 **HashMap** 將每次查找降為 $O(1)$ ，才是更有效率的方法。



- 若用線性搜尋，每次查找是 $O(n)$ ，整體 $O(n^2)$ ；若用 HashMap 為每個字母維護獨立的 stack，每次查找是 $O(1)$ ，整體 $O(n)$ 。

這個對比清楚展示了 HashMap 在需要快速查找某個元素是否存在，並存取其歷史記錄的場景下的效率，也讓我更理解與熟悉 Hash。

方法	Time	Space
HashMap+Stack	$O(n)$	$O(n)$
Stack線搜	$O(n^2)$	$O(n)$



```

1 class Solution(object):
2     # 解法一：HashMap + 各字母獨立 Stack O(n)
3     from collections import defaultdict
4     def calculateScore_hashmap(self, s):
5         # 每個字母各自維護一個 stack，確保配對時取的是最近出現的索引
6         # 用 defaultdict 避免每次存取前需要判斷 key 是否存在
7         stacks = defaultdict(list)
8         score = 0
9         for i, c in enumerate(s):
10            # a 對應 z、b 對應 y，第 i 個字母對應第 25-i 個
11            # ord(c)-ord('a') 將字元轉為 0~25，25 減掉後再轉回字元
12            mirror = chr(ord('z') - (ord(c) - ord('a')))
13            if stacks[mirror]:
14                # pop 取出最近出現的鏡像索引
15                # 選 stack 而非 queue 是因為最近出現的優先配對
16                score += i - stacks[mirror].pop()
17            else:
18                stacks[c].append(i)
19        return score
20

```

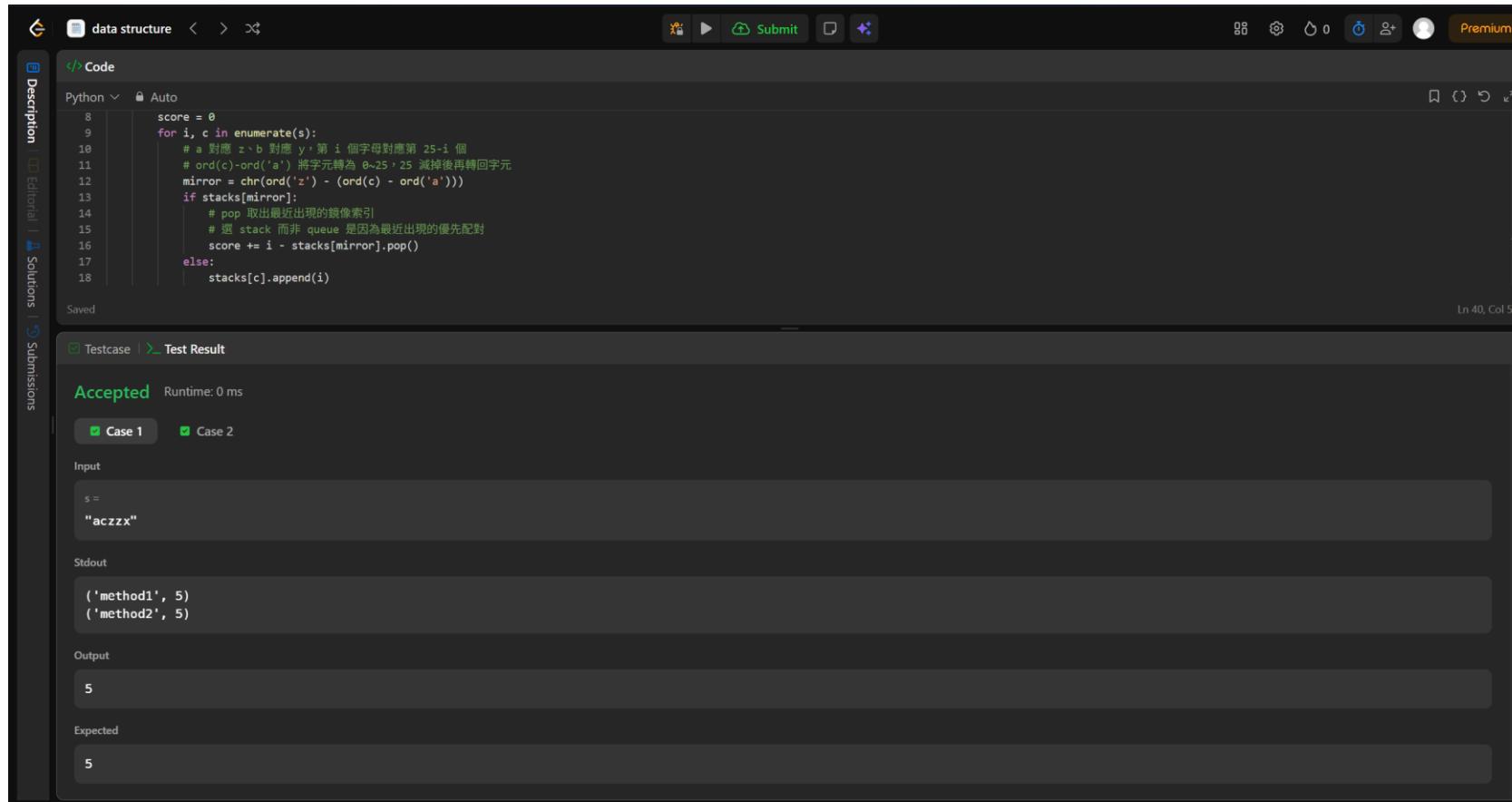
```

21 # 解法二：單 Stack 線搜 (O(n^2)，用於對比)
22 def calculateScore_naive(self, s):
23     stack = []
24     score = 0
25     for i, c in enumerate(s):
26         mirror = chr(ord('z') - (ord(c) - ord('a')))
27         found = -1
28         # 從後往前搜尋是為了找最近出現的鏡像
29         # 但每次都要遍歷整個 stack，導致整體 O(n^2)
30         for j in range(len(stack)-1, -1, -1):
31             if stack[j][0] == mirror:
32                 found = j
33                 break
34         if found != -1:
35             score += i - stack[found][1]
36             stack.pop(found)
37         else:
38             stack.append((c, i))
39     return score
40
41 def calculateScore(self, s):
42     res1 = self.calculateScore_hashmap(s)
43     res2 = self.calculateScore_naive(s)
44
45     print("method1", res1)
46     print("method2", res2)
47
48     return res1

```



- 結果證明：



The screenshot shows a code editor interface with a Python script and its test results. The script is a solution for a problem involving a string and a stack. The test results show that the solution is accepted for two test cases.

```

Python 3.9.7 Auto
8     score = 0
9     for i, c in enumerate(s):
10        # a 對應 z, b 對應 y, 第 i 個字母對應第 25-i 個
11        # ord(c)-ord('a') 將字元轉為 0~25, 25 減掉後再轉回字元
12        mirror = chr(ord('z') - (ord(c) - ord('a'))))
13        if stacks[mirror]:
14            # pop 取出最近出現的鏡像索引
15            # 題 stack 而非 queue 是因為最近出現的優先配對
16            score += i - stacks[mirror].pop()
17        else:
18            stacks[c].append(i)

```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

s =
"aczzx"

Stdout

('method1', 5)
('method2', 5)

Output

5

Expected

5



LeetCode編碼：

779

系所： 北聯(北醫) 生物醫學工程學系
學號： B812111001
姓名： 張博翔





定義一個無限延伸的字元序列生成規則：

第 1 行只有一個字元 0；

從第 2 行開始，每一行由上一行生成，規則是將上一行的每個 0 替換為 01，每個 1 替換為 10。

給定第 n 行與位置 k 。

- 前幾行展開如下：第 1 行為 0；第 2 行由 0 展開為 01；第 3 行由 01 展開為 0110；第 4 行由 0110 展開為 01101001
- 第 n 行第 k 個字元由上一行第 $\lfloor k/2 \rfloor$ 個字元生成，
奇數位置的子字元與父字元相同，偶數位置的子字元與父字元相反。

遞迴解法的核心，避免搞混奇偶對應規則。遞迴的終止條件是 $n = 1$ ，此時唯一的字元固定為 0。



- 解法一：遞迴找父節點
- 第 n 行第 k 個字元是由第 $n-1$ 行第 $\lfloor k/2 \rfloor$ 個字元生成的。因為展開規則是每個字元變成兩個字元，所以第 n 行的第 k 個字元一定來自上一行的第 $\lfloor k/2 \rfloor$ 。
- 父子節點之間的關係取決於 k 的奇偶性。
每個父字元展開後，奇數位與父字元相同，偶數位與父字元相反。這是因為 $0 \rightarrow 01$ 與 $1 \rightarrow 10$
因此只需要不斷向上追蹤父節點，直到第 1 行，再沿路計算每一層的翻轉狀況，就能得到答案。
- 遞迴深度為 n ，時間複雜度 $O(n)$ ，空間複雜度 $O(n)$ (call stack)。
優點是邏輯直觀易懂。缺點是 n 愈大時 call stack 愈深，且每層都需要一次遞迴調用。



- 解法二：位元運算 $O(\log k)$
- 將遞迴的過程轉化成數學公式，直接由 k 的二進位表示計算答案，時間複雜度降至 $O(\log k)$ ，且不需要遞迴或迭代。
- 觀察每一層的「是否翻轉」恰好對應 $k-1$ 的二進位中每一個 **bit** 的值。
 從第 1 行的 0 出發，每往下一層，若當前位置是偶數位（右邊的子字元）則翻轉一次，若是奇數位則不翻轉。
 判斷每層是奇數位還是偶數位，等同於看 $k-1$ 的對應 **bit** 是 0 還是 1。因此從第 1 行走到第 n 行的過程中，總共翻轉的次數等於 $\text{bin}(k-1)$ 中 1-bit 的個數。
 翻轉偶數次等同於沒有翻轉，結果為 0；翻轉奇數次結果為 1。
- 優點是 $O(\log k)$ 時間、 $O(1)$ 空間。缺點是規律本身需要透過數學推導才能理解，若沒有推導過程，直接看程式碼 `bin(k-1).count('1') % 2` 可讀性低。



- 第 n 行第 k 個字元的值由從根到該位置的路徑上翻轉了幾次決定。不需要真正展開整個序列，只需要追蹤翻轉次數的奇偶性。
而翻轉次數恰好對應 $k-1$ 的二進位中 **1-bit** 的個數，這讓問題從需要逐層遞迴，變成一個純粹的位元計算。
把遞迴規律壓縮成位元運算的思維，可以應用在 **LC 3304** 中。

方法	Time	Space
遞迴	$O(n)$	$O(n)$
位元運算	$O(\log k)$	$O(1)$



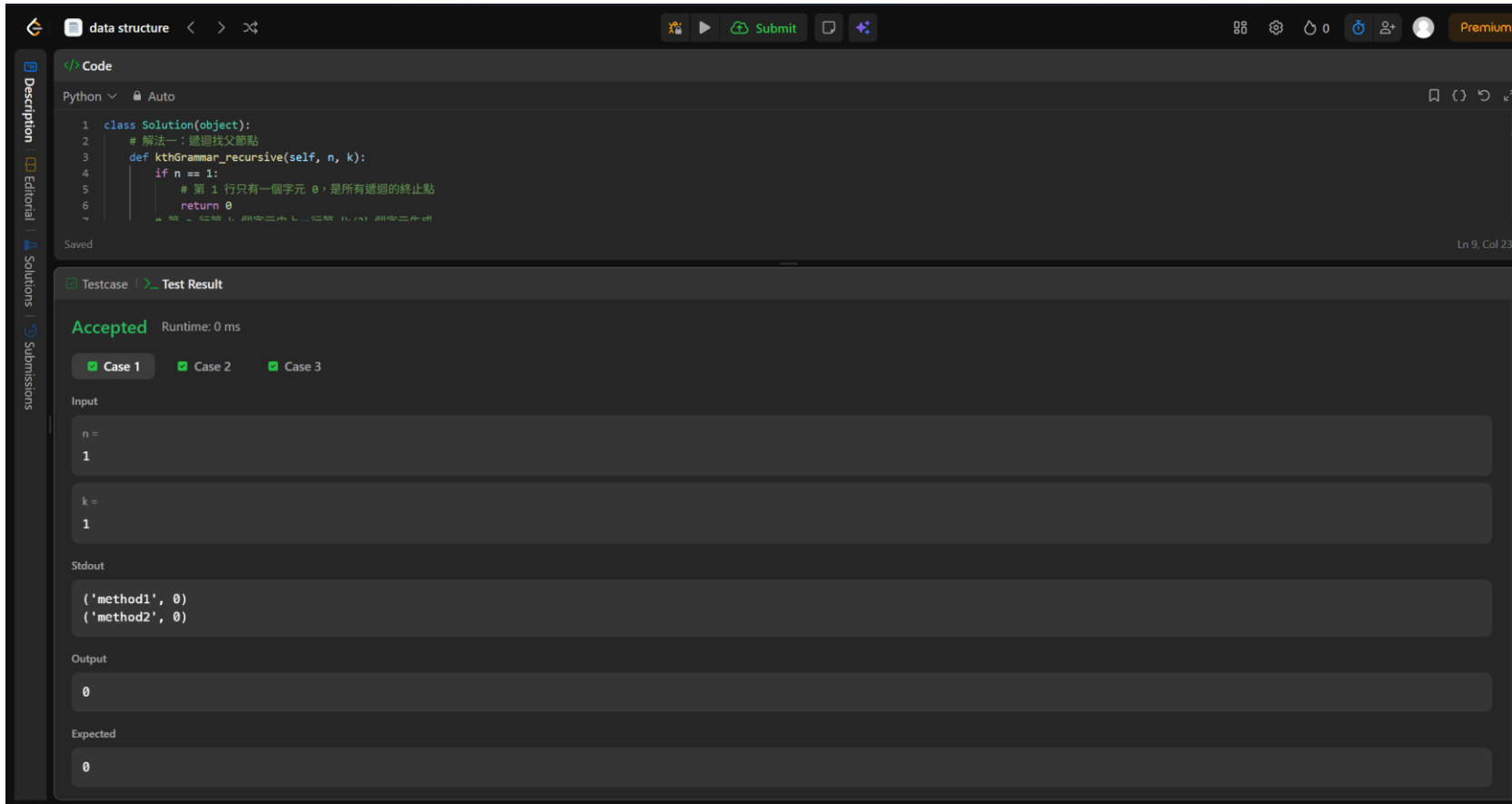
```

1 class Solution(object):
2     # 解法一：遞迴找父節點
3     def kthGrammar_recursive(self, n, k):
4         if n == 1:
5             # 第 1 行只有一個字元 0，是所有遞迴的終止點
6             return 0
7         # 第 n 行第 k 個字元由上一行第 [k/2] 個字元生成
8         # (k+1)//2 是 [k/2] 的整數寫法，避免浮點運算
9         parent = self.kthGrammar_recursive(n-1, (k+1)//2)
10        if k % 2 == 1:
11            # k 為奇數代表是父字元展開後的左邊那個（繼承父字元）
12            return parent
13        else:
14            # k 為偶數代表是父字元展開後的右邊那個（翻轉父字元）
15            return 1 - parent
16
17    # 解法二：位元運算 O(log k)
18    def kthGrammar_bits(self, n, k):
19        # k-1 的二進位中每個 bit 對應一層的「是否翻轉」
20        # bit 為 1 代表在後半段（被翻轉），bit 為 0 代表在前半段（不翻轉）
21        # 翻轉奇數次結果為 1，偶數次結果為 0，取模 2 即為答案
22        return bin(k-1).count('1') % 2
23
24    def kthGrammar(self, n, k):
25        res1 = self.kthGrammar_recursive(n, k)
26        res2 = self.kthGrammar_bits(n, k)
27
28        print("method1", res1)
29        print("method2", res2)
30
31        return res1

```



- 結果證明：



The screenshot shows a code editor with the following Python code:

```

1 class Solution(object):
2     # 解法一：遞迴找父節點
3     def kthGrammar_recursive(self, n, k):
4         if n == 1:
5             # 第 1 行只有一個字元 0，是所有遞迴的終止點
6             return 0

```

The test result section shows:

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

n = 1

k = 1

Stdout

('method1', 0)
('method2', 0)

Output

0

Expected

0



LeetCode編碼：

3304

系所：北聯(北醫) 生物醫學工程學系
學號：B812111001
姓名：張博翔



題目 (含解釋&說明)



從只包含一個字元 'a' 的字串開始，每一輪操作將當前字串中每個字元都替換為其字母表後繼字元（ $a \rightarrow b$ 、 $b \rightarrow c$ 、...、 $z \rightarrow a$ ，循環），並將產生的新字串附加到原字串尾端，使字串長度翻倍。

給定位置 k （1-indexed），求足夠多輪操作後第 k 個字元為何。

- 前幾輪展開如下：
 初始為 "a"；第 1 輪將 a 後繼為 b，附加後變成 "ab"；
 第 2 輪將 ab 後繼為 bc，"abbc"；
 第 3 輪將 abbc 後繼為 bccd，"abbcbbccd"。
- 字母後繼有循環，z 的後繼回到 a，計算時需要對 26 取模。
 題目保證 $k \leq 500$ ，因此模擬法最多展開 9 輪（ $2^9 = 512 \geq 500$ ）就足夠。



- 解法一：字串展開
- 從 "a" 開始，每輪將當前字串中每個字元都往後移一位，產生新字串後附加到尾端，直到字串長度達到或超過 k ，再取第 $k-1$ 個字元（轉換為 0-indexed）回傳。
- 字元後移的計算方式是 $\text{chr}((\text{ord}(c) - \text{ord}('a') + 1) \% 26 + \text{ord}('a'))$ ，先將字元轉為 0~25 的數字，加 1 後對 26 取模確保 z 能循環回 a ，再轉回字元。
- 缺點是字串每輪翻倍， k 愈大時記憶體與時間消耗會指數成長，無法推廣極端情況。



- 解法二：位元運算 $O(\log k)$
- 方法與 LC 779 的位元解法相似，相同的數學結構。
觀察展開的過程可以發現，每個字元的值由它被「後移」了幾次決定。
- 在前半還是後半的判斷，對應 $k-1$ 的二進位中每一個 bit：bit 為 1 代表在後半段（被後移過），bit 為 0 代表在前半段（未被後移）。
因此從初始的 'a' 出發，總共被後移的次數等於 $\text{bin}(k-1)$ 中 1-bit 的個數，最終字元為 $\text{chr}(\text{ord}('a') + \text{shifts})$ （不需要取模，因為 $k \leq 500$ 時 shifts 最多為 9，不會超過 'z'）
- 779 中每層翻轉的次數決定最終是 0 或 1（奇偶性）；3304 中每層後移的次數累加後決定最終字元（直接加總）。
兩題都是把展開過程壓縮成 $k-1$ 的位元表示。



方法	Time	Space
字串展開	$O(k)$	$O(k)$
位元運算	$O(\log k)$	$O(1)$



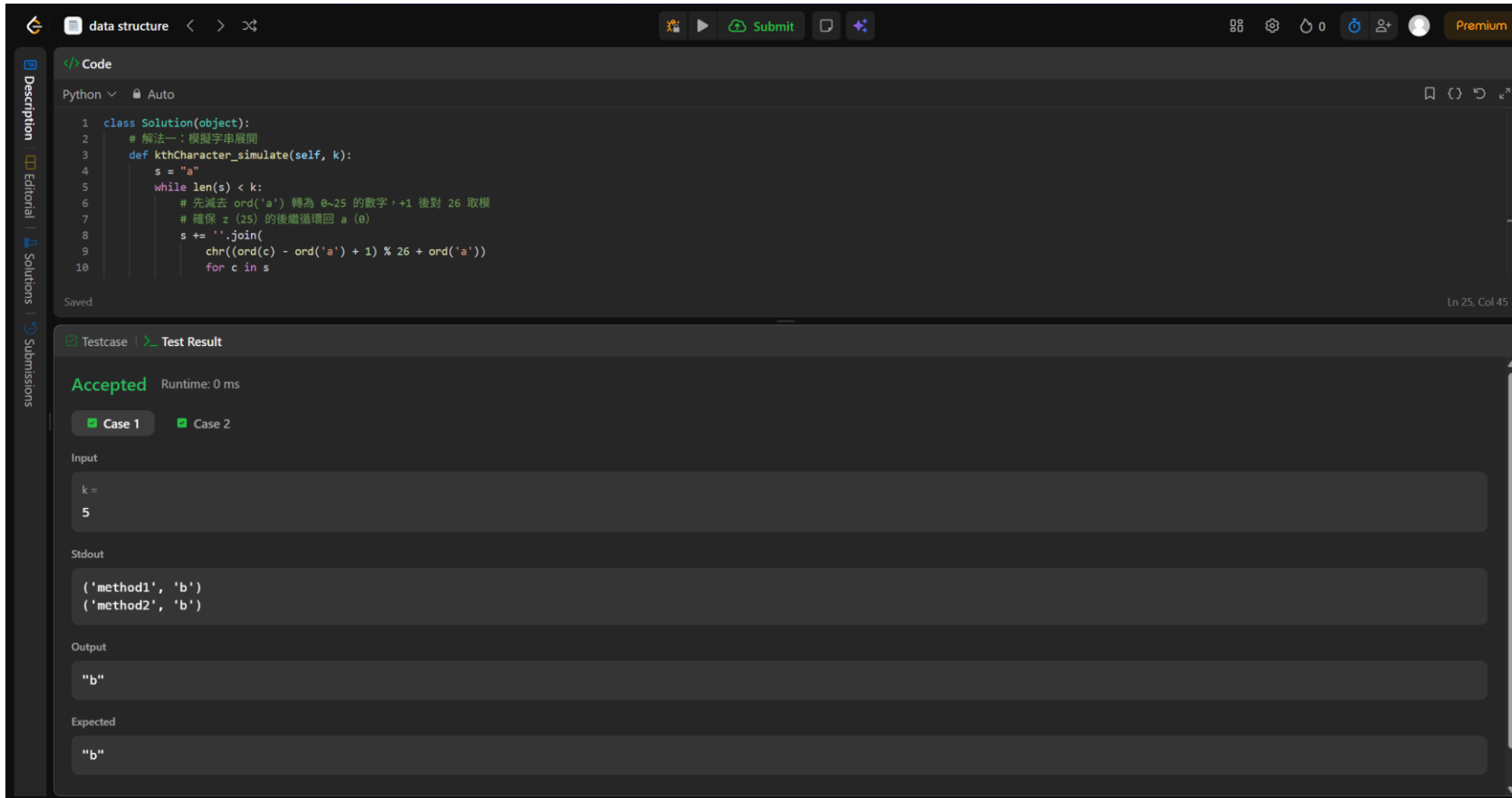
```

1 class Solution(object):
2     # 解法一：模擬字串展開
3     def kthCharacter_simulate(self, k):
4         s = "a"
5         while len(s) < k:
6             # 先減去 ord('a') 轉為 0~25 的數字，+1 後對 26 取模
7             # 確保 z (25) 的後繼循環回 a (0)
8             s += ''.join(
9                 chr((ord(c) - ord('a') + 1) % 26 + ord('a'))
10                for c in s
11            )
12         # 轉換為 0-indexed 取得第 k 個字元
13         return s[k-1]
14
15     # 解法二：位元運算 O(log k)
16     def kthCharacter_bits(self, k):
17         # k-1 的二進位中每個 bit 對應一層的「是否後移」
18         # bit 為 1 代表在後半段（被後移一次），bit 為 0 代表在前半段
19         # 與 LC 779 不同的是，這裡累加後移次數而非看奇偶性
20         shifts = bin(k-1).count('1')
21         # 從 'a' 開始後移 shifts 次，不需取模因為 k≤500 時 shifts 最多為 9
22         return chr(ord('a') + shifts)
23
24     def kthCharacter(self, k):
25         res1 = self.kthCharacter_simulate(k)
26         res2 = self.kthCharacter_bits(k)
27
28         print("method1", res1)
29         print("method2", res2)
30
31         return res1
32

```



- 結果證明：



```

1 class Solution(object):
2     # 解法一：模擬字串展開
3     def kthCharacter_simulate(self, k):
4         s = "a"
5         while len(s) < k:
6             # 先減去 ord('a') 轉為 0~25 的數字 +1 後對 26 取模
7             # 確保 z (25) 的後繼循環回 a (0)
8             s += ''.join(
9                 chr((ord(c) - ord('a') + 1) % 26 + ord('a'))
10                for c in s

```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

k =
5

Stdout

('method1', 'b')
('method2', 'b')

Output

"b"

Expected

"b"